Auto MPG Prediction Using KNN

A machine learning web application that predicts vehicle fuel efficiency (Miles Per Gallon) using the K-Nearest Neighbors **(KNN) algorithm—turning raw 1970s–80s automotive data into actionable insights.**

Web App

Interactive interface for exploring and predicting MPG

Dataset

398 vehicle records from the classic Auto MPG dataset

Model

K-Nearest Neighbors regression for fuel efficiency prediction

Performance

Strong final fit with **R^2 = 0.9013**

This project is valuable because it combines a clean web experience with a reliable predictive model, making it easier to understand how vehicle characteristics influence fuel efficiency and to estimate MPG with strong accuracy.

View on GitHub

About Me

> Narinder Singh — pursuing a diploma at Lamrin Tech Skills University, Ropar, with industrial training at NIELIT Ropar. Passionate about machine learning and data science, driven to turn data into meaningful solutions.

I am currently building my technical foundation through my diploma studies at **Lamrin Tech Skills University, Ropar** and hands-on industrial training at **NIELIT Ropar**. My academic and practical experience has helped me develop a strong interest in **machine learning** and **data science**, with a focus on solving real-world problems through data-driven approaches.

My technical expertise includes working with analytical tools, exploring datasets, and understanding how models can be used to generate meaningful predictions and insights. I am especially motivated by the opportunity to learn continuously, apply new technologies, and build solutions that are both practical and impactful.

My career goal is to grow as a skilled data professional who can contribute to innovative projects in machine learning and analytics. I am driven by curiosity, problem-solving, and the desire to transform data into useful outcomes that support better decisions.

What We'll Cover

Explore the journey from raw data to a predictive web application.

1

Dataset Overview

Understanding the automotive data.

2

Algorithm Explanation

Deep dive into K-Nearest Neighbors.

3

Data Preprocessing

Cleaning and transforming data for ML.

4

Model Optimization

Fine-tuning for best performance.

5

Performance Results

Evaluating accuracy and insights.

6

Deployment

Bringing the model to a web app.

Auto MPG Dataset — UCI Repository

Real-world vehicle specifications from the UCI Machine Learning Repository, collected from cars in the 1970s–1980s. It's a classic supervised learning dataset for predicting fuel efficiency.

398

Rows

Vehicle records

8

Features

Engine, cylinders, displacement, horsepower, weight, acceleration, model year, origin

1

Target

MPG — the value to predict

8 Key Features

Cylinders

Engine cylinders tied to power and efficiency.

Displacement

Engine size in cubic inches.

Horsepower

Engine output affecting performance and fuel use.

Weight

Vehicle mass, a major MPG factor.

Acceleration

Time needed to reach speed.

Model Year

Manufacturing year and design changes.

Origin

Country of manufacture.

MPG (Target)

The fuel-efficiency value to predict.

Key Input Features

Cylinders
Core indicator of engine size

Horsepower
Higher power = lower MPG

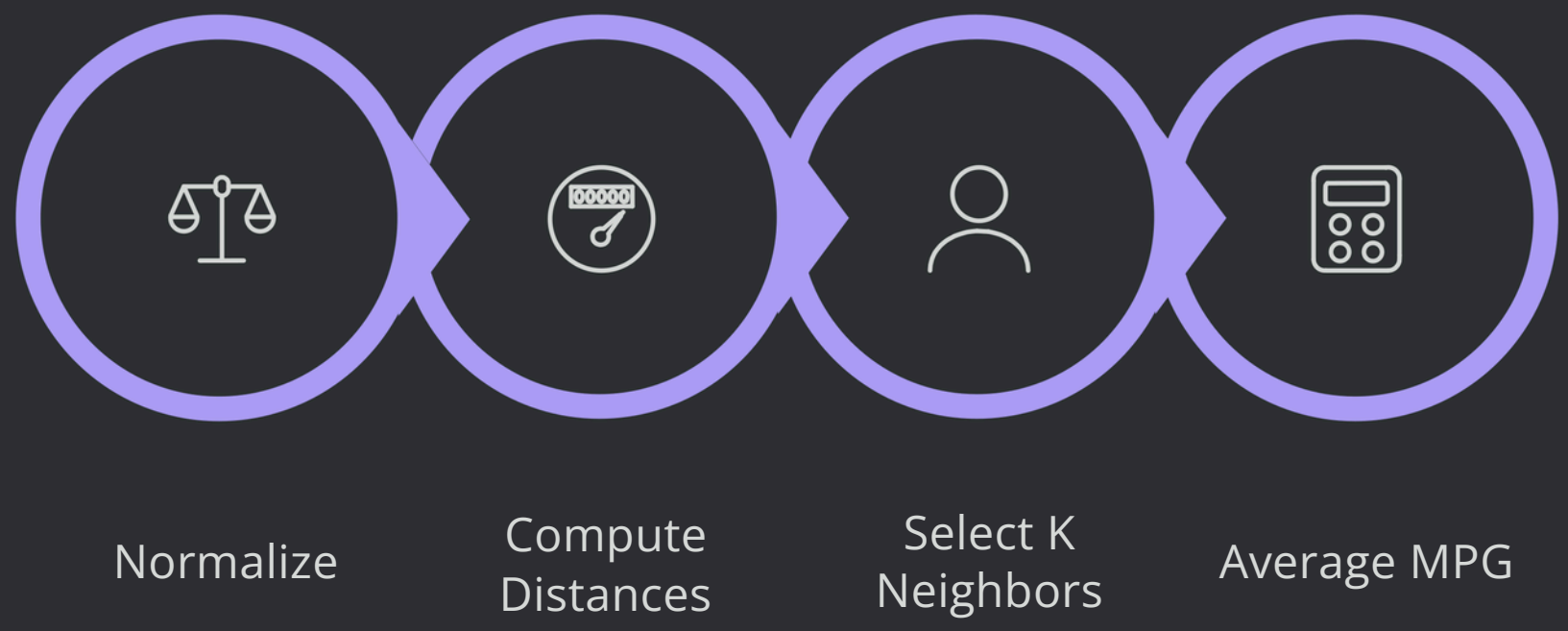
Displacement
Correlated with fuel consumption

Weight
Strongest predictor of efficiency

Model Year
New models = moreefficient

How KNN Works

KNN is a distance-based algorithm: it predicts a new vehicle's MPG by finding the training examples that are most similar in Feature space. Similarity is measured by distance, so points that are closer together are treated as more alike.



Euclidean distance is the straight-line distance between two points in feature space. If two vehicles have similar values for cylinders, horsepower, displacement, and weight, their distance will be small; if they differ a lot, their distance will be large.

Normalization is critical because KNN is sensitive to scale. Without it, a feature like weight or displacement can dominate the distance calculation simply because it has larger numeric values, even if another feature is more predictive. Standardizing or scaling features ensures each variable contributes more fairly.

Step-by-step example:

1. A new vehicle enters with features such as 4 cylinders, 110 horsepower, 2.0 displacement, and 2,800 lb weight.
2. The model normalizes those values using the same scaling applied to the training data.
3. It computes Euclidean distance from that vehicle to every labeled training example.
4. It sorts the results and selects the **K nearest neighbors**.
5. It predicts MPG by averaging the neighbors' MPG values, or sometimes by taking a weighted average so closer neighbors count more.

The choice of **K** strongly affects the result. A small K, such as 1 or 3, makes predictions very sensitive to individual points and can capture local detail, but it may overfit noise. A larger K smooths predictions and reduces noise, but it can oversimplify the boundary and miss important local patterns.

This is the classic **bias-variance tradeoff**: small K tends to have low bias and high variance, while large K tends to have higher bias and lower variance. In practice, K is usually selected by validation so the model balances responsiveness and stability.

Practical implications: KNN is easy to explain and can work well when similar vehicles have similar MPG, but it can be expensive at prediction time because it must compare the new example against many training points. It also works best when features are meaningful, scaled properly, and not too noisy or redundant.

Data Preprocessing Pipeline

1

Handle Missing Values

Why it's necessary: KNN depends on distance calculations, and missing values break or distort those distances.

Problem solved: The Auto MPG dataset uses "?" for missing horsepower values, which must be converted to a numeric form before modeling.

Method: Replace "?" with NaN and then impute the median with fillna():

```
df["horsepower"] = df["horsepower"].replace("?", np.nan)
df["horsepower"] = df["horsepower"].astype(float)
df["horsepower"] = df["horsepower"].fillna(df["horsepower"].median())
```

Impact on performance: Preserves more training examples, avoids dropping rows unnecessarily, and reduces the influence of outliers compared with mean imputation.

2

Feature Selection

Why it's necessary: KNN works best when the input space contains relevant, informative features and does not include identifiers or noisy text fields.

Problem solved: The target is MPG; car name is dropped because it is not a numeric predictor and would not help distance-based similarity.

Method: Keep the core numeric predictors: cylinders, displacement, horsepower, weight, acceleration, model year, and origin

```
X = df[["cylinders", "displacement", "horsepower", "weight",
        "acceleration", "model year", "origin"]]
y = df["mpg"]
```

Impact on performance: Removes irrelevant columns, reduces noise, and improves distance comparisons by focusing KNN on features tied to fuel efficiency.

3

Feature Scaling

Why it's necessary: KNN is highly sensitive to feature scale, because large-magnitude variables can dominate Euclidean distance.

Problem solved: Without scaling, weight or displacement could overwhelm smaller-scale variables like acceleration.

Method: Apply StandardScaler to standardize each feature to mean=0 and std=1:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Impact on performance: Makes each feature contribute more fairly to the distance metric, which usually improves KNN accuracy and prevents one variable from dominating neighbor selection.

4

Train/Test Split

Why it's necessary: A separate test set is needed to measure how well the model generalizes to unseen vehicles.

Problem solved: Evaluating on the training set would give an overly optimistic result and hide overfitting.

Method: Use train_test_split with an 80/20 split and random_state=42 for reproducibility:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

Impact on performance: Provides an unbiased estimate of test performance and makes experiments repeatable, so model comparisons are consistent.

5

Model Training:

Why it's necessary: The KNN model must be fit on the training data so it can learn how examples relate to one another in the feature space.

Problem solved: Training establishes the learned neighbors and distance relationships that KNN uses to make future predictions.

Method: Use scikit-learn's KNeighborsRegressor with k=9 and fit it on the training set:

```
from sklearn.neighbors import KNeighborsRegressor

knn = KNeighborsRegressor(n_neighbors=9)
knn.fit(X_train, y_train)
```

Impact on performance: Creates the foundation for making predictions on new data by storing the training examples and defining the neighbor-based decision rule.

Model Performance: Before & After Optimization

BASELINE K=5

R^2 = 0.7660 | MAE = 2.77 MPG

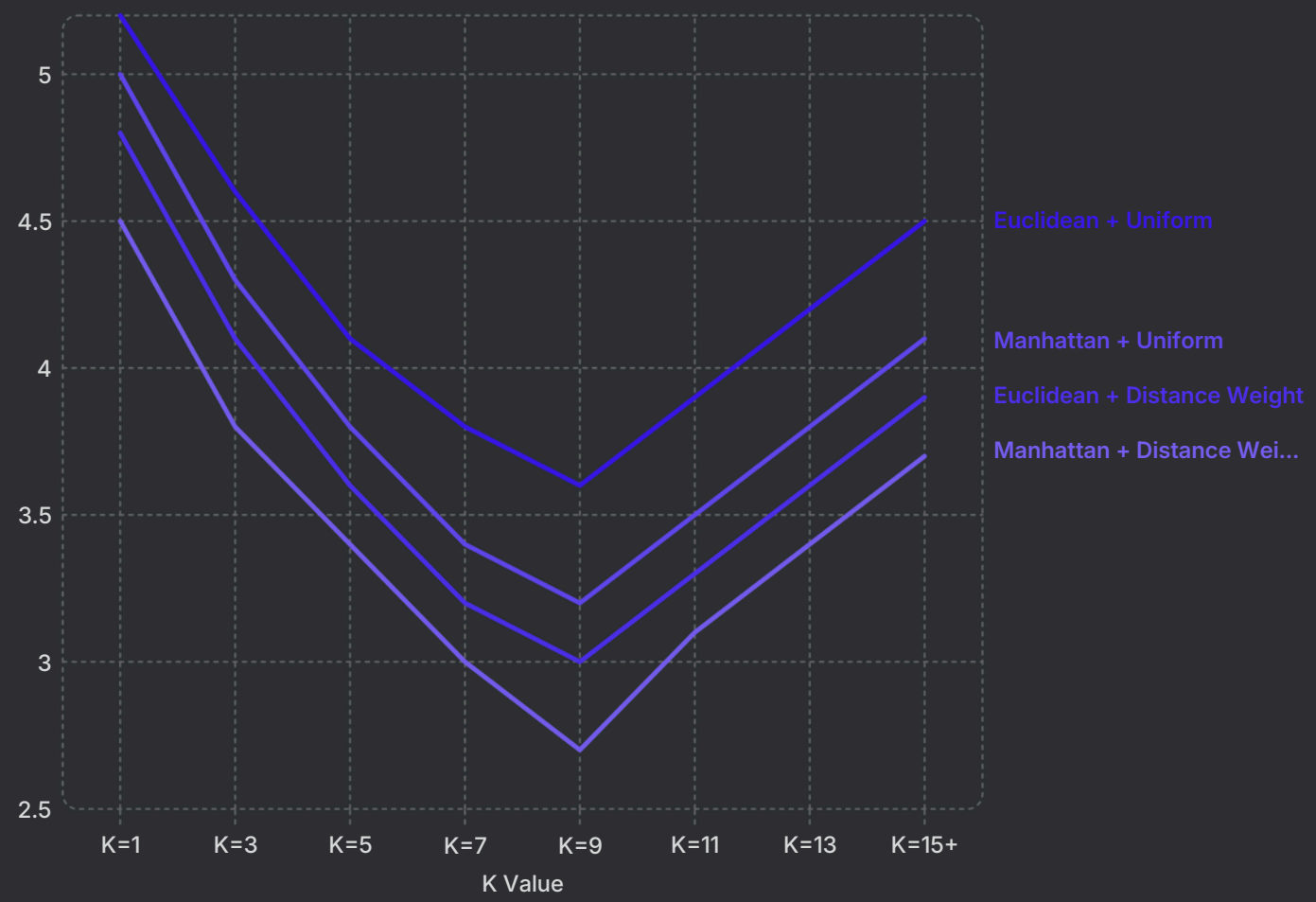
Significant scatter — weak correlation between actual and predicted values.

OPTIMIZED K=9

R^2 = 0.9013 | MAE = 1.76 MPG

Points tightly clustered — optimal config: K=9, distance weighting, Manhattan metric.

Optimal K = 9: Best Hyperparameter Choice



Key Findings

Distance Weight + Manhattan consistently yields the lowest MAE across all K values.

Small K (K=1)

High variance — overfits to noise

Optimal K=9

Best bias-variance balance

Large K

High bias — misses local patterns

Flask & HTML: Serving the Model

Flask (Backend)

Lightweight Python framework bridging the ML model and browser. `@app.route('/')` maps URLs to Python functions.

HTML (Frontend)

Jinja2 templates render input forms (cylinders, horsepower, weight) and display predicted MPG dynamically.

Submit Form → Flask POST → Run KNN → Render Result

Tech Stack & Results

Python
Core language

Pandas
Data manipulation

Scikit-learn
KNN & evaluation

Matplotlib
Visualization

90.13%

Explains most MPG variance and shows a strong model fit.

1.76

Average prediction error is low, usually within about 2 MPG.

K=9

Balanced accuracy and stability for the best KNN setting.

Test Drive the Model Yourself!

Experience the power of the KNN prediction model firsthand. Use our interactive web application to input car features and get instant MPG predictions.

Launch Interactive Demo